



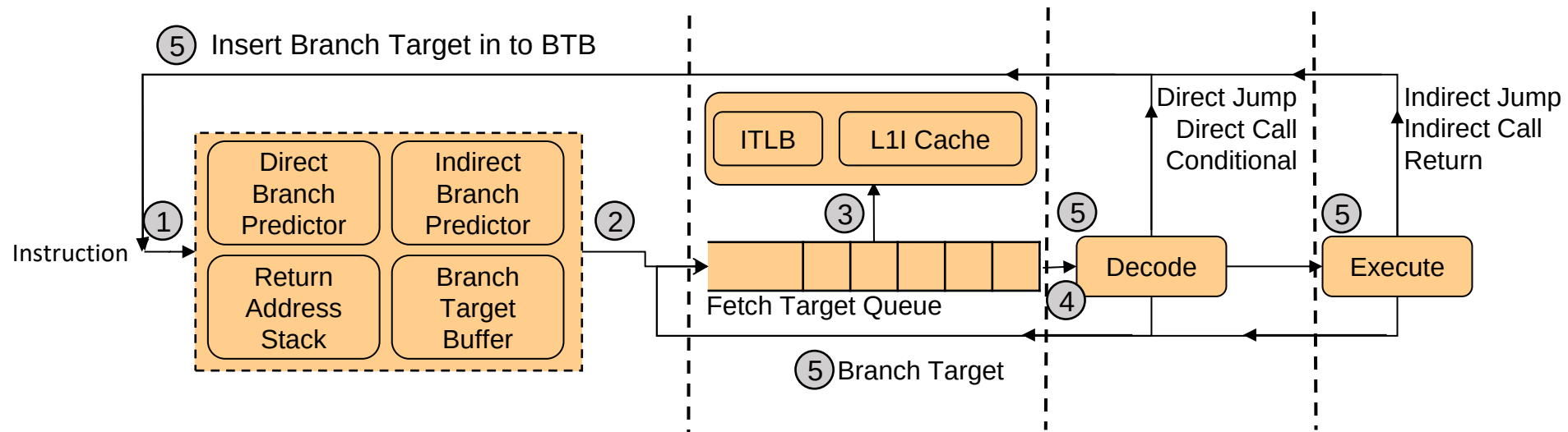
CS683: Advanced Computer Architecture

Lecture 18: Thunderstorms, hmmm, still there 😊
Front-end Mysteries, hmmm, de-mystified

<https://www.cse.iitb.ac.in/~biswa/courses/>

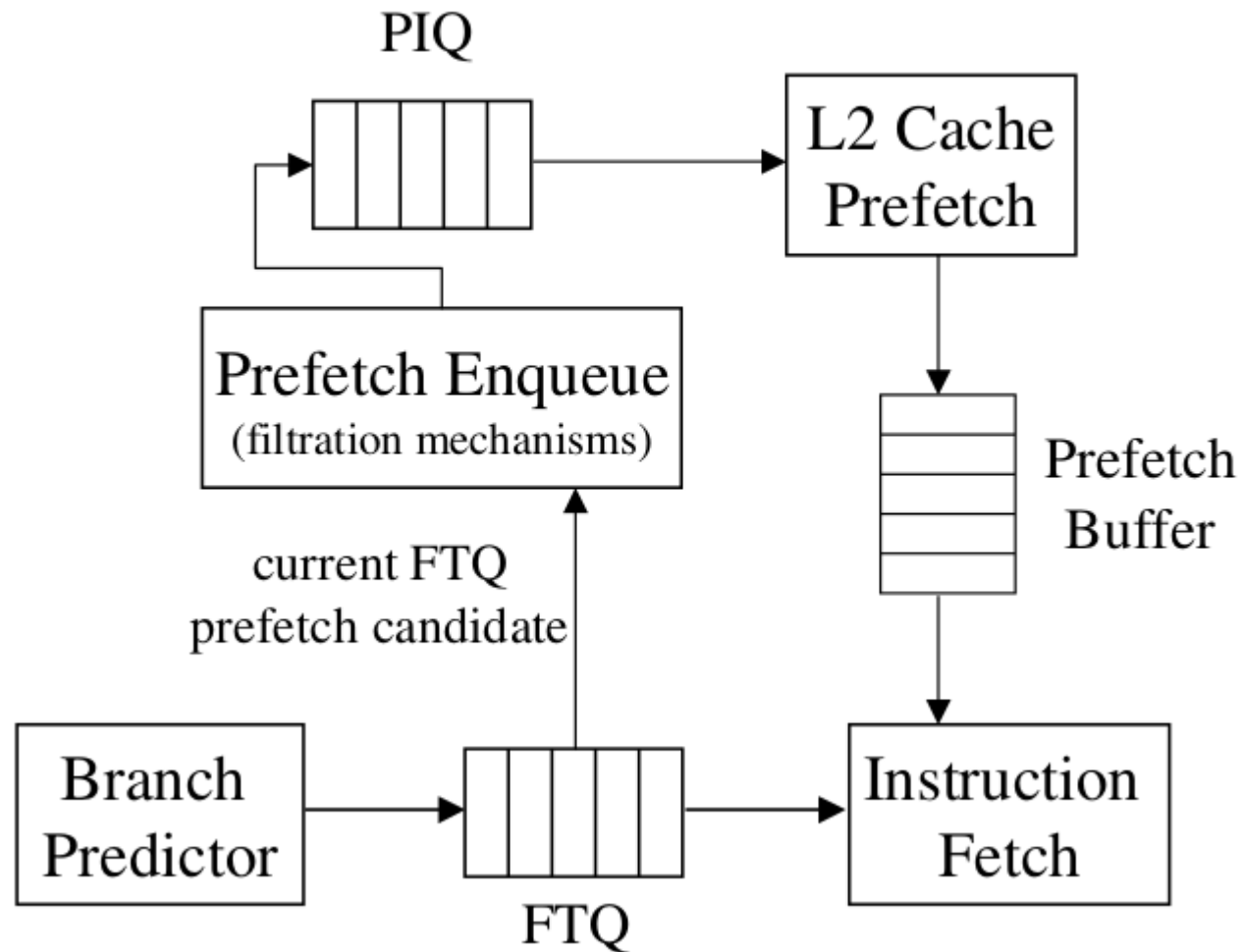
<https://www.cse.iitb.ac.in/~biswa/>

Decoupled Front-end



Branch predictor looks ahead even if L1I misses

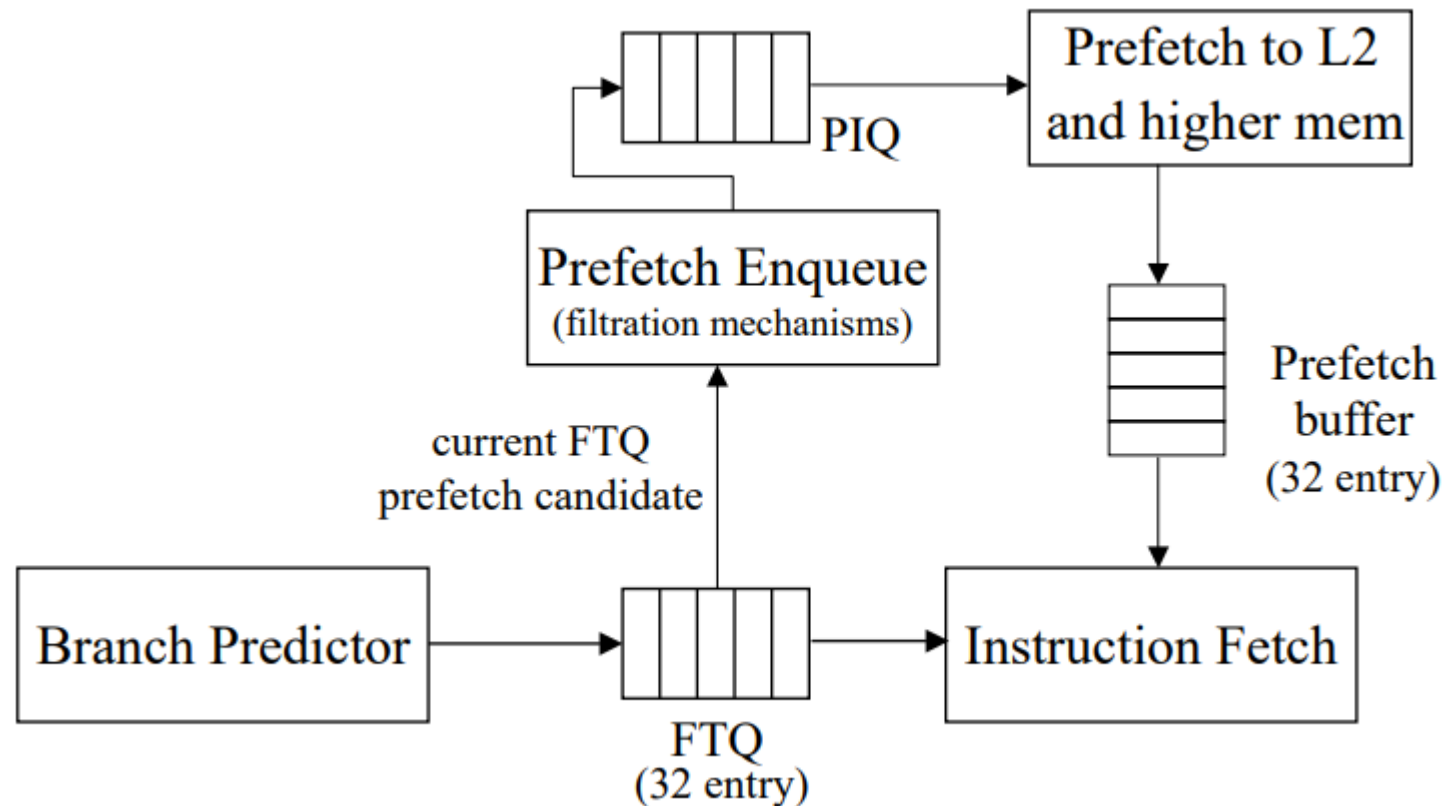
FDIP(Fetch Directed Instruction Prefetching)



PIQ: Prefetch Instruction Queue
FTQ: Fetch Target Queue

FTQ directed code prefetching (FDIP)

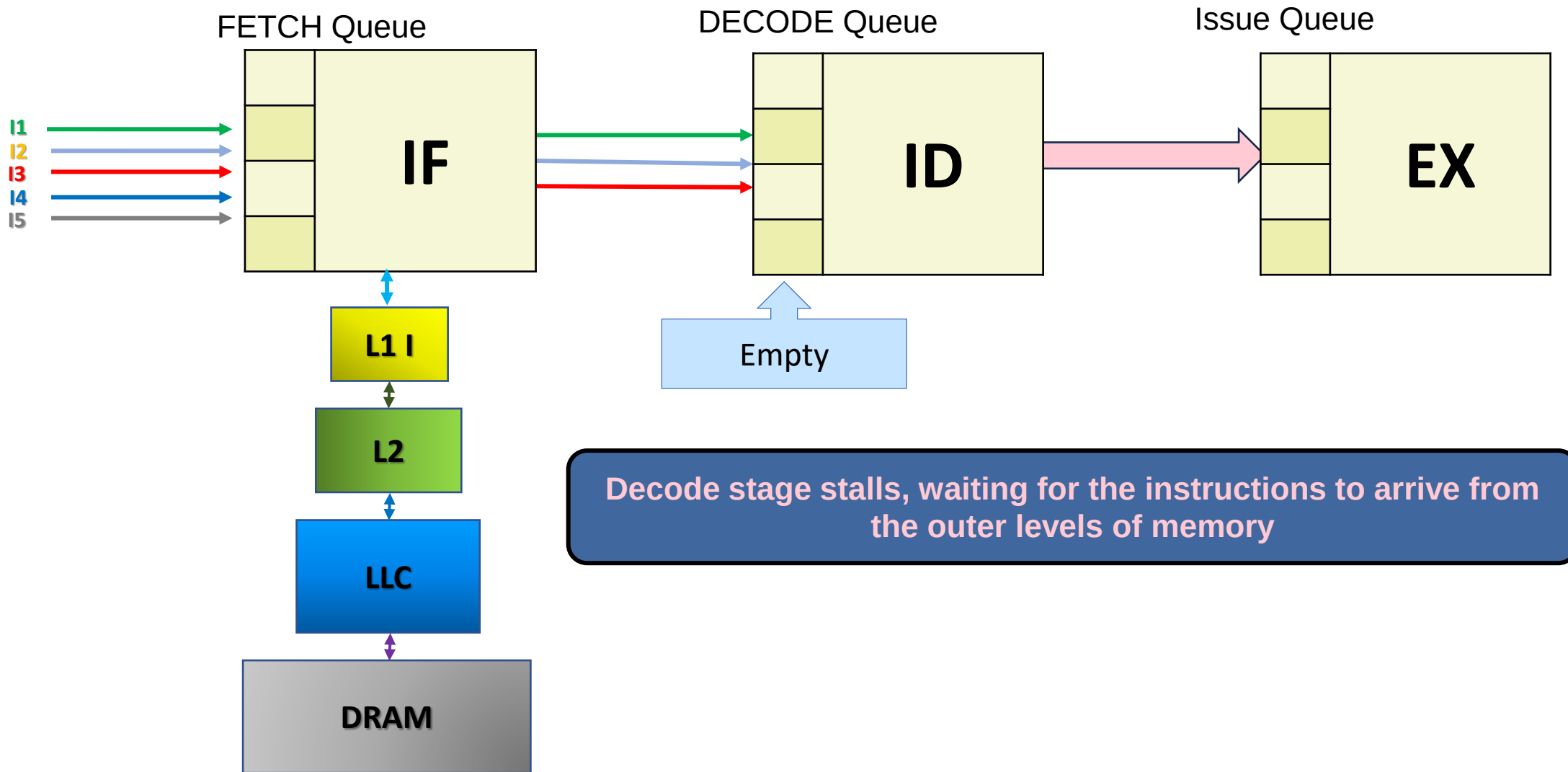
One entry in FTQ stores 8 instructions



Benefits

- Branch predictor can move ahead and L1I misses can be amortized 😊
- Instruction prefetching?
- Hides latency of multi-level branch predictor and BTB
- Demands a larger BTB though 😞
- BTB prefetching?
- Tradeoffs: BTB capacity vs L1I capacity and latency
- What else?

L1I misses can cause Decode Starvation



Decode Latency will be there, anyways 😞

- x86 uses variable-length instructions
- Size of the decoder cannot be fixed like ARM ISA.
- Also, x86 is RISCY ISA 😊 CISC for programmers, RISC for the execution units
- One CISC instruction -> many RISC micro-operations
- Here is the deal: There is a macro-op queue containing CISC instructions that a **pre-decoder** logic converts into micro-ops. It can take up to six cycles in the worst case.
- Then there will a decode. In general, decoding takes a considerable amount of power and latency.
- What can be done? Well, use a cache, called micro-op cache. A hit in the micro-op cache can bypass decode latency.
- BTW, which stage in the pipeline, the micro-op cache will be accessed?

Think about latency and bandwidth again

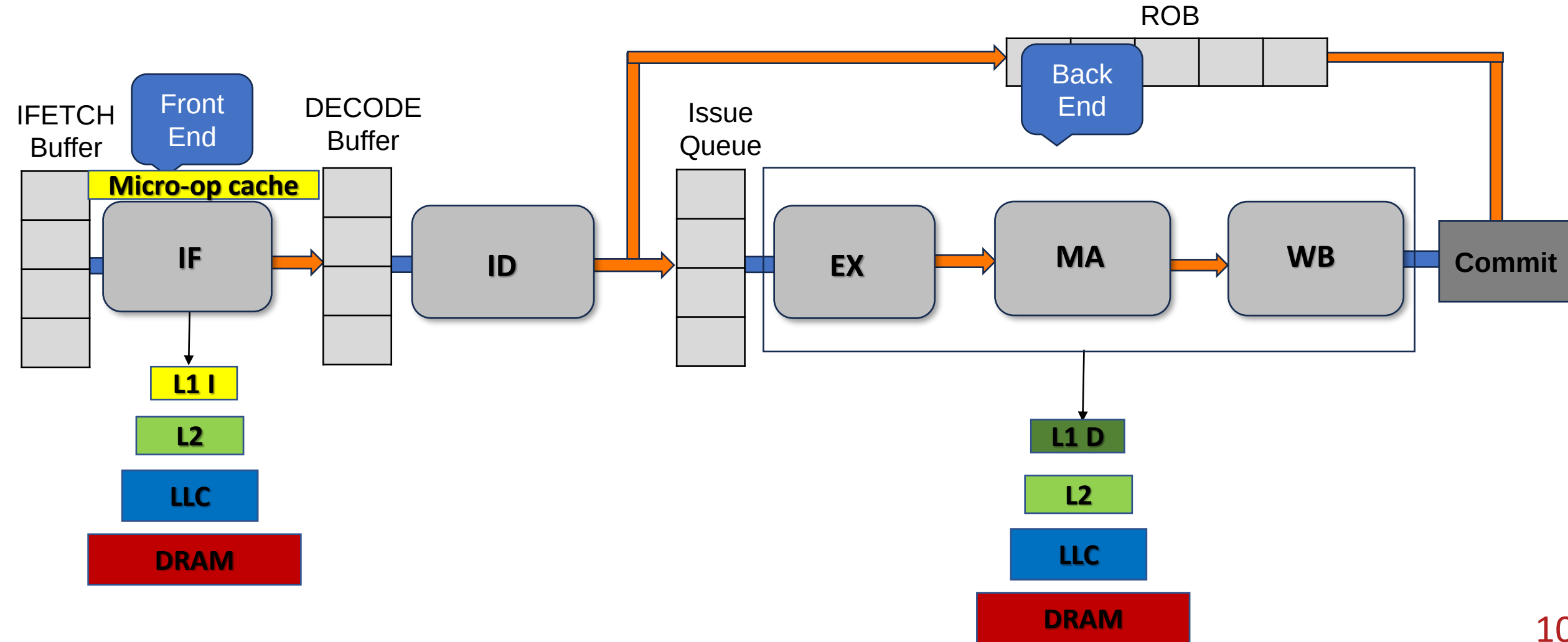
- Front-end latency and bandwidth
- Bad speculation (branch predictor, micro-op cache too 😊)

Students@CS683
after Front-end
mysteries



We are together:

The Bigger Picture (latency and bandwidth)



O3 (out-of-order) processor

- Pipelining: Tries to achieve $CPI = 1$
- Hardware scheduling so far: In order execution Instructions after stall must wait even if independent.

Dynamic scheduling: Out of order execution Hardware lookahead of blocked instructions

- Inorder, O3
- Inorder fetch, O3 execute, Inorder completion

Issue Stage??

- To enable O3 execution,

We split Decode stage into

- I. Dispatch: Decode instructions, check for structural hazards
- II. Issue: Read operands after all the data hazards are taken care of and issue the instructions to next stage

Issue Q (instructions enter in-order, leave out-of-order)

